Week 3 — Functions Deep Dive & Pythonic Patterns

Priority: HIGH | Interview Hot Topic

---

1. Generators & `yield`

What is a generator?

A generator is a function that **pauses and resumes** instead of running start-to-finish and returning once. Any function containing `yield` becomes a generator function — calling it doesn't run the code, it returns a **generator object**.

```
def count_up(n):
    i = 1
    while i <= n:
        yield i      # pause here, send value out
        i += 1

gen = count_up(3)   # nothing has run yet
print(next(gen))    # 1
print(next(gen))    # 2
print(next(gen))    # 3
print(next(gen))    # raises StopIteration
```

`yield` vs `return`

`return`	`yield`
Ends function, sends one value, state is destroyed	Pauses function, sends one value, **state is preserved**
Function can only be called/run once per call	Function can be resumed many times via `next()`
JS equivalent: normal function return	JS equivalent: `function*` generator with `yield`

Why use generators? (Memory efficiency)

A list builds the **entire result in memory**. A generator produces values **one at a time, on demand (lazy evaluation)**.

```
# BAD for large data — loads everything into RAM
def get_squares_list(n):
    return [x*x for x in range(n)]

# GOOD — computes one value at a time
def get_squares_gen(n):
    for x in range(n):
        yield x*x
```

This matters massively when working with large files, streams, or infinite sequences.

Practical example: reading a large file line by line

This is the exact "what to practice" task from the slide:

```
def read_large_file(file_path):
    with open(file_path, 'r') as f:
        for line in f:
            yield line.strip()

# Usage — only one line is in memory at a time
for line in read_large_file("huge_log.txt"):
    if "ERROR" in line:
        print(line)
```

Compare to `f.readlines()`, which loads the **whole file** into a list first — bad for multi-GB files.

Generator expressions (genexpr)

Same syntax as a list comprehension, but `()` instead of `[]` — lazy, not stored in memory.

```
squares_list = [x*x for x in range(1000000)]   # built immediately, uses RAM
squares_gen  = (x*x for x in range(1000000))   # built lazily, near-zero RAM

sum(x*x for x in range(1000000))  # most common real use — feed straight into
sum/max/any/etc.
```

The iterator protocol (what's happening under the hood)

- `next(gen)` resumes the generator until the next `yield` or until it ends.
- When it ends, Python raises `StopIteration` automatically (loops handle this for you).
- A `for` loop over a generator is just repeatedly calling `next()` and catching `StopIteration`.

Advanced (good to know, rarely asked at junior level)

```
gen.send(value)   # resume generator AND send a value INTO it (received by `x =
yield`)
gen.throw(exc)    # raise an exception inside the generator at the paused point
gen.close()       # stop the generator early
```

Quick interview answer

> "A generator is a function that yields values lazily, one at a time, maintaining state between calls — it doesn't compute everything upfront like a list does. Great for large datasets, streams, and infinite sequences."

---

2. Lambdas & Higher-Order Functions

Lambda syntax

A `lambda` is an **anonymous, single-expression function**. No `def`, no name (unless assigned), no

block of statements — just one expression that is automatically returned.

```
square = lambda x: x * x
print(square(5))   # 25

add = lambda a, b: a + b
print(add(2, 3))   # 5
```

JS equivalent: `(x) => x * x`

Lambda vs `def` — when to use which

Use `lambda`	Use `def`
Quick, throwaway, single-expression logic	Anything with multiple lines/statements
Passed inline as an argument (`key=`, `map`, `filter`)	Needs a docstring or is reused elsewhere
No `if/elif/else` blocks, no loops, no multiple statements	Needs error handling, logging, etc.

```
# Good lambda use — inline sort key
people = [("Ali", 25), ("Sara", 19), ("Zain", 31)]
people.sort(key=lambda person: person[1])

# Bad lambda use — too complex, should be a real function
# lambda x: x*2 if x > 0 else (x*3 if x < -10 else 0)   # avoid this
```

Functions are first-class objects in Python

This is the foundation of higher-order functions. Functions can be:

1. Assigned to a variable
2. Passed as an argument to another function
3. Returned from another function

```
def greet():
    return "hello"

f = greet          # 1. assigned to variable (no parentheses!)
print(f())         # "hello"

def shout(func):   # 2. passed as argument
    return func().upper()

print(shout(greet))  # "HELLO"

def make_multiplier(n):   # 3. returned from a function (closure)
    def multiplier(x):
        return x * n
    return multiplier

times3 = make_multiplier(3)
print(times3(10))  # 30
```

The core higher-order functions

`map(func, iterable)` — applies `func` to every item, returns an iterator

```
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, nums))   # [2, 4, 6, 8]
```

`filter(func, iterable)` — keeps items where `func` returns `True`

```
evens = list(filter(lambda x: x % 2 == 0, nums))   # [2, 4]
```

`functools.reduce(func, iterable)` — folds the iterable into a single value

```
from functools import reduce
total = reduce(lambda acc, x: acc + x, nums)   # 10
```

`sorted(iterable, key=func)` — most common real-world use of lambdas

```
words = ["banana", "kiwi", "apple"]
sorted_by_len = sorted(words, key=lambda w: len(w))   # ['kiwi', 'apple',
'banana']
```

> **Pythonic note:** `map`/`filter` exist, but Python culture usually prefers **list comprehensions** for readability:

```
doubled = [x * 2 for x in nums]                # preferred over map()
evens   = [x for x in nums if x % 2 == 0]       # preferred over filter()
```

`reduce` and `sorted(key=...)` are still genuinely idiomatic and commonly used.

---

3. Decorators

The core idea

A decorator is **a function that takes a function and returns a new function**, usually wrapping extra behavior around the original.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before the function runs")
        result = func(*args, **kwargs)
        print("After the function runs")
        return result
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
# Before the function runs
# Hello!
# After the function runs
```

`@decorator` is just syntactic sugar

```
@my_decorator
def say_hello():
    ...

# is EXACTLY equivalent to:
def say_hello():
    ...
say_hello = my_decorator(say_hello)
```

This is why the slide says "no built-in decorator syntax" in JS — JS has no `@` equivalent natively; you'd manually wrap: `sayHello = withLogging(sayHello)`.

Always use `*args, **kwargs` in the wrapper

This makes the decorator reusable on **any** function signature, not just one with no arguments.

```
def timer(func):
    import time
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f"{func.__name__} took {end - start:.4f}s")
        return result
    return wrapper
```

`functools.wraps` — don't skip this

Without it, the wrapped function loses its original name/docstring (becomes `wrapper` instead of the real function name). This matters for debugging, introspection, and frameworks like Flask/FastAPI that read `__name__`.

```
from functools import wraps

def timer(func):
    @wraps(func)   # preserves func.__name__, func.__doc__, etc.
    def wrapper(*args, **kwargs):
        ...
        return func(*args, **kwargs)
    return wrapper
```

Decorators WITH arguments (decorator factories)

This is the trickiest part — needed for `@retry(max_attempts=3)`. You need **three layers** of nested functions:

```
def retry(max_attempts=3):          # layer 1: takes the decorator's OWN
arguments
    def decorator(func):            # layer 2: takes the function being
decorated
        @wraps(func)
        def wrapper(*args, **kwargs):   # layer 3: the actual call
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"Attempt {attempt} failed: {e}")
```

```
                    if attempt == max_attempts:
                        raise
        return wrapper
    return decorator

@retry(max_attempts=3)
def call_api():
    ...
```

Mental model:

- `retry(max_attempts=3)` runs immediately → returns `decorator`
- `decorator` is then applied to `call_api` → returns `wrapper`
- `call_api` now actually points to `wrapper`

Decorator stacking — `@a @b def f(): ...`

Stacked decorators apply **bottom-up**, wrapping outward:

```
@a
@b
def f():
    ...

# is equivalent to:
f = a(b(f))
```

`b` wraps `f` first, then `a` wraps the result of that. So execution order when calling `f()`: **`a`'s "before" code → `b`'s "before" code → `f` itself → `b`'s "after" code → `a`'s "after" code.**

```
@timer
@retry(max_attempts=3)
def call_api():
    ...
# = timer(retry(max_attempts=3)(call_api))
# Order matters: timer measures the WHOLE retry process (including all failed
attempts)
```

Closures + `nonlocal` (what makes decorators possible)

A closure is an inner function that "remembers" variables from its enclosing scope, even after the outer function has finished running. This is exactly how `wrapper` still has access to `func` and `max_attempts`.

```
def counter():
    count = 0
    def increment():
        nonlocal count   # without this, count += 1 would raise
UnboundLocalError
        count += 1
        return count
    return increment

c = counter()
print(c())  # 1
print(c())  # 2
```

JS equivalent: closures work the same way, but JS doesn't need `nonlocal` — `let`/regular variable assignment in a closure just works because JS doesn't have Python's local-scope-write restriction.

Bonus: `functools.lru_cache` — a real built-in decorator

Worth knowing as a practical example of decorators used in production:

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)
```

---

4. Functional Programming Basics

Core principles

1. **Pure functions** — same input always gives same output, no side effects (doesn't modify external state, doesn't print/write files as part of its logic).
2. **Immutability** — prefer creating new data over mutating existing data in place.
3. **First-class & higher-order functions** — covered above (functions as values).
4. **Function composition** — building complex behavior by combining small functions (decorators are a form of this).

```
# Impure (has a side effect, depends on external state)
total = 0
def add_to_total(x):
    global total
    total += x

# Pure (same input -> same output, no side effects)
def add(a, b):
    return a + b
```

Comprehensions as Python's functional style

Python favors comprehensions over chained `map`/`filter` for readability — this is the Pythonic idiom:

```
# Functional style (more JS-like)
result = list(map(lambda x: x*2, filter(lambda x: x % 2 == 0, range(10))))

# Pythonic style (preferred)
result = [x*2 for x in range(10) if x % 2 == 0]
```

Key `functools` tools to know

Function	Purpose
`reduce(func, iterable)`	Fold a sequence into a single value
`partial(func, *args)`	Pre-fill some arguments of a function, get a new function back
`lru_cache` / `cache`	Memoize a function's results automatically
`wraps`	Preserve metadata when writing decorators

```
from functools import partial

def power(base, exp):
    return base ** exp

square = partial(power, exp=2)
print(square(5))   # 25
```

---

5. JS → Python Quick Reference

JavaScript	Python
No direct equivalent	`yield` — pauses a function, returns a generator
`(x) => x * x`	`lambda x: x * x` (single expression only — no statements/blocks)
Higher-order functions exist (`map`, `filter`, `reduce`)	Same concept exists, but **comprehensions are preferred**
No built-in decorator syntax (manual wrapping: `f = withLogging(f)`)	`@decorator` — syntactic sugar for `f = decorator(f)`
Closures work similarly; just reference outer variable	Closures work similarly; need `nonlocal` to **reassign** an outer variable (not needed to just read it)
`function*` + `yield` for generators	`def` + `yield` for generators
Array spread `...args`	`*args` (positional) and `**kwargs` (keyword)

---

6. What to Practice (from the slide) — Worked Through

Write a generator that reads a large file line by line → see Section 1 (`read_large_file`)

Build 3 custom decorators: timing, logging, retry

```
from functools import wraps
import time

def timer(func):
    @wraps(func)
```

```
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"[TIMER] {func.__name__} took {time.perf_counter()-start:.4f}s")
        return result
    return wrapper

def logger(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"[LOG] Calling {func.__name__} with args={args},
kwargs={kwargs}")
        result = func(*args, **kwargs)
        print(f"[LOG] {func.__name__} returned {result}")
        return result
    return wrapper

def retry(max_attempts=3, delay=1):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"[RETRY] Attempt {attempt}/{max_attempts} failed:
{e}")
                    if attempt == max_attempts:
                        raise
                    time.sleep(delay)
        return wrapper
    return decorator
```

Understand decorator stacking: `@a @b def f(): ...` means `a(b(f))` → see Section 3

---

7. Mini-Project: `@retry(max_attempts=3)` + `@timer` Applied to an API-Calling Function

```
import time
import random
from functools import wraps

# --- Decorators ---

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            duration = time.perf_counter() - start
            print(f"[TIMER] '{func.__name__}' finished in {duration:.4f}s")
    return wrapper

def retry(max_attempts=3, delay=1, exceptions=(Exception,)):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last_exception = None
```

```
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    last_exception = e
                    print(f"[RETRY] Attempt {attempt}/{max_attempts} failed:
{e}")
                    if attempt < max_attempts:
                        time.sleep(delay)
            raise last_exception
        return wrapper
    return decorator

# --- Simulated API call ---

class APIError(Exception):
    pass

@timer
@retry(max_attempts=3, delay=0.5, exceptions=(APIError,))
def call_external_api(endpoint: str):
    """Simulates an unreliable API that fails ~60% of the time."""
    if random.random() < 0.6:
        raise APIError(f"Failed to reach {endpoint}")
    return {"status": "success", "endpoint": endpoint}

# --- Usage ---
if __name__ == "__main__":
    try:
        response = call_external_api("https://api.example.com/data")
        print("Final result:", response)
    except APIError as e:
        print("All retries exhausted:", e)
```

What's happening, stack order:

- `@timer` is applied last (outermost) → it measures the **entire retry loop**, not just one attempt.
- `@retry(...)` is applied first (innermost) → it directly wraps `call_external_api`.
- Execution order: `timer.wrapper()` starts the clock → calls `retry.wrapper()` → which tries `call_external_api()` up to 3 times → `timer.wrapper()` stops the clock once retry logic finishes (success or final failure).

Why `finally` in `timer`: ensures the duration is printed even if all retries fail and an exception propagates up.

Interview-Ready Summary

Concept	**One-line definition**
Generator	Function using `yield` that produces values lazily, pausing/resuming state
Lambda	Anonymous, single-expression function
Higher-order function	A function that takes and/or returns other

Concept	**One-line definition**
	functions
Decorator	A function that wraps another function to add behavior, applied via `@`
Decorator factory	A function that returns a decorator, allowing the decorator to take arguments
Closure	An inner function retaining access to its enclosing scope's variables
Pure function	Same inputs → same outputs, no side effects